



ITESO, Universidad  
Jesuita de Guadalajara

Reproducción de ataque a dispositivo móvil

Presentado por:  
Leal Telles Carlos Alberto

Presentado para:  
SocialTIC

# Indice

|                                                                    |          |
|--------------------------------------------------------------------|----------|
| <b>Reproducción de ataque a dispositivo móvil.....</b>             | <b>1</b> |
| <b>Indice.....</b>                                                 | <b>2</b> |
| <b>1. Introducción.....</b>                                        | <b>3</b> |
| <b>2. Contexto de la Vulnerabilidad.....</b>                       | <b>3</b> |
| <b>3. Explicación Técnica de la Vulnerabilidad.....</b>            | <b>4</b> |
| 3.1 El Formato ABX (Android Binary XML).....                       | 4        |
| 3.2 El Error: writeShort() y el Desbordamiento de Entero.....      | 4        |
| 3.3 La Inyección del stageDir.....                                 | 5        |
| 3.4 La Primitiva de Escritura.....                                 | 5        |
| 3.5 La Escalada a SYSTEM.....                                      | 5        |
| 3.6 El Crash Final: Señal SIGSEGV.....                             | 6        |
| <b>4. Herramientas para la Recreación.....</b>                     | <b>6</b> |
| 4.1 Emulador de Android Studio.....                                | 6        |
| 4.2 ADB (Android Debug Bridge).....                                | 6        |
| 4.3 AndroidQF (Android Quick Forensics).....                       | 6        |
| 4.4 MVT (Mobile Verification Toolkit).....                         | 6        |
| 4.5 Repositorio del Exploit.....                                   | 6        |
| <b>5. Proceso de Recreación del Ataque.....</b>                    | <b>7</b> |
| 5.1 Preparación del Entorno.....                                   | 7        |
| Paso 1: Clonar el repositorio del exploit con:.....                | 7        |
| Paso 2: Abrir el proyecto en Android Studio.....                   | 7        |
| Paso 3: Configurar las versiones correctas de herramientas.....    | 7        |
| Paso 4: Compilar el APK.....                                       | 7        |
| 5.2 Configuración del Emulador Vulnerable.....                     | 7        |
| Paso 1: Crear el emulador.....                                     | 7        |
| Paso 2: Iniciar el emulador y verificar conectividad ADB con:..... | 8        |
| Paso 3: Conceder permisos de root al ADB con:.....                 | 8        |
| 5.3 Ejecución del Exploit.....                                     | 8        |
| Paso 1: Instalar la aplicación maliciosa con.....                  | 8        |
| Paso 2: Ejecutar el exploit.....                                   | 8        |
| Paso 3: Observar el crash y reinicio del sistema.....              | 8        |
| 5.4 Verificación de la Escalada de Privilegios.....                | 8        |
| Paso 1: Identificar el proceso de la aplicación maliciosa.....     | 8        |
| Paso 2: Verificar el UID del proceso.....                          | 8        |
| Paso 3: Verificar el payload instalado.....                        | 8        |
| <b>6. Proceso Forense.....</b>                                     | <b>9</b> |
| 6.1 Extracción con AndroidQF.....                                  | 9        |
| 6.2 Analisis con MVT.....                                          | 10       |
| Paso 1: Transferir las extracciones a Kali Linux.....              | 10       |

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| Paso 2: Activar el entorno virtual de MVT.....                                  | 10 |
| Paso 3: Analizar la extracción pre-ataque.....                                  | 11 |
| Paso 4: Analizar la extracción post-ataque.....                                 | 11 |
| 7. Hallazgos del Análisis Forense.....                                          | 11 |
| • Archivo: command.log (Registro de Ejecución).....                             | 12 |
| Hallazgo 1.1 – Paquetes no oficiales detectados.....                            | 12 |
| Hallazgo 1.2 – Configuración de seguridad debilitada.....                       | 12 |
| Hallazgo 1.3 – Binarios de root detectados.....                                 | 12 |
| • Archivo: tombstones.json (Volcados de Memoria).....                           | 12 |
| Hallazgo 2.1 – Crash del system_server (tombstone_00).....                      | 12 |
| • Archivo: aqf_packages_detected.json (Paquetes Instalados encontrados por MVT) | 13 |
| Hallazgo 3.1 – Aplicación maliciosa principal.....                              | 13 |
| Hallazgo 3.2 – Payload malicioso con privilegios SYSTEM.....                    | 14 |
| • Archivo: bugreport_timestamps.json (Marcas de Tiempo del Sistema).....        | 14 |
| Hallazgo 4.1 – Perfiles de aplicación de las apps maliciosas.....               | 14 |
| Hallazgo 4.2 – Tombstones generados durante el ataque.....                      | 15 |
| Hallazgo 4.3 – Archivos ANR (Application Not Responding).....                   | 15 |
| 8. Discusión y Conclusiones.....                                                | 15 |
| 8.1 Indicadores de Compromiso (IoCs).....                                       | 15 |
| 8.2 Timeline.....                                                               | 16 |
| 8.3 Tácticas, Técnicas y Procedimientos (TTPs).....                             | 17 |
| 8.4 Conclusiones.....                                                           | 19 |
| 9. Referencias.....                                                             | 19 |

# 1. Introducción

Este documento tiene como objetivo documentar el proceso de explotación de la vulnerabilidad CVE-2024-34740 (AbxOverflow) y el posterior análisis forense de sus rastros en un dispositivo Android. Siguiendo la metodología de documentación Diátaxis, el contenido se organiza en dos modos principales:

- How-to Guide: Secciones 4, 5 y 6, que proporcionan instrucciones paso a paso para recrear el ataque y realizar el análisis forense.
- Explanation: Secciones 2, 3, 7 y 8, que explican en profundidad el contexto técnico de la vulnerabilidad, los hallazgos forenses y su interpretación.

El ataque se basa en el CVE-2024-34740, una vulnerabilidad de desbordamiento de entero en BinaryXmlSerializer de Android, que permite a una aplicación sin permisos especiales escalar a privilegios de SYSTEM (UID 1000). La práctica se realizó en un emulador de Android Studio con una imagen vulnerable (Android 13, parche de seguridad 2024-03-01), utilizando herramientas forenses como AndroidQF y MVT para extraer y analizar evidencias antes y después del ataque.

## 2. Contexto de la Vulnerabilidad

CVE-2024-34740 es una vulnerabilidad de desbordamiento de entero (integer overflow) en la clase BinaryXmlSerializer de Android, específicamente en los métodos `attributeBytesBase64()` y `attributeBytesHex()`. Esta clase es responsable de serializar arrays de bytes en archivos ABX (Android Binary XML), un formato binario utilizado por Android a partir de la versión 12 para almacenar archivos de estado del sistema con el fin de mejorar el rendimiento.

El impacto de la vulnerabilidad es crítico, ya que permite que una aplicación maliciosa sin permisos especiales escale a privilegios de SYSTEM (UID 1000). La vulnerabilidad fue parcheada por Google en octubre de 2024, y afecta a las versiones de Android 12, 13 y 14 con niveles de parche anteriores a esa fecha.

La vulnerabilidad fue descubierta y explotada activamente en la naturaleza, siendo utilizada por la aplicación maliciosa de Pinduoduo para escalar privilegios en millones de dispositivos. Fue añadida al catálogo de CISA Known Exploited Vulnerabilities (KEV) en marzo de 2023.

## 3. Explicación Técnica de la Vulnerabilidad

### 3.1 El Formato ABX (Android Binary XML)

A partir de Android 12, el sistema comenzó a utilizar un formato binario (ABX) para almacenar archivos de estado de servicios críticos que se ejecutan en el proceso `system_server`. Esta decisión se tomó por razones de rendimiento, ya que la lectura y escritura de estos archivos

consumía alrededor del 1.5% del tiempo total de CPU del sistema. Algunos de estos archivos, ubicados en /data/system/, incluyen:

- /data/system/packages.xml : Registro maestro de todas las aplicaciones instaladas, sus UUIDs, permisos y firmas. Es el objetivo final del exploit.
- /data/system/install\_sessions.xml : Almacena el estado de las sesiones de instalación de aplicaciones en curso. Es el vector de inyección del exploit.

### 3.2 El Error: writeShort() y el Desbordamiento de Entero

La vulnerabilidad reside en la implementación de los métodos attributeBytesBase64() y attributeBytesHex() de la clase BinaryXmlSerializer. Estos métodos serializan arrays de bytes dentro del formato ABX. El problema está en cómo escriben la longitud del array:

```
@Override
public XmlSerializer attributeBytesBase64(String namespace, String name, byte[] value)
    throws IOException {
    if (namespace != null && !namespace.isEmpty()) throw illegalNamespace();
    mOut.writeByte(ATTRIBUTE | TYPE_BYTES_BASE64);
    mOut.writeInternedUTF(name);
    mOut.writeShort(value.length);
    mOut.write(value);
    return this;
}
```

El método writeShort() tiene un límite de 65535 ( $2^{16} - 1$ ). Si se proporciona un array de exactamente 65536 bytes, se produce un desbordamiento de entero: el valor 65536 supera el límite y se almacena como 0. Sin embargo, a continuación, el método escribe los 65536 bytes de datos reales. Cuando el parser (BinaryXmlPullParser) lee el archivo, se encuentra con un atributo que, según su cabecera, tiene longitud 0, pero inmediatamente después hay 65536 bytes de datos. Esto corrompe la estructura del XML, permitiendo al atacante inyectar su propio contenido.

### 3.3 La Inyección del stageDir

El exploit elige como objetivo el archivo /data/system/install\_sessions.xml porque cualquier aplicación, sin necesidad de permisos especiales, puede crear una sesión de instalación mediante la API PackageManager.Session. El proceso de inyección es el siguiente:

1. **Creación de la sesión:** La app maliciosa crea una sesión de instalación normal.
2. **Inyección del Payload:** Utiliza el método setChecksums() de la sesión para proporcionar un array de bytes (los checksums) de exactamente 65536 bytes de longitud. Este array es el payload malicioso.
3. **Forzar la Escritura:** El exploit crea inmediatamente una segunda sesión de instalación. Esta acción fuerza al system\_server a escribir el estado de todas las sesiones (incluyendo la nuestra con el payload) en el archivo install\_sessions.xml.

El payload construido en AbxInjector.java tiene la siguiente estructura:

- Cierra la etiqueta actual: Inyecta un token END\_TAG para cerrar el elemento <session> legítimo que contiene el checksum corrupto.
- Abre una nueva etiqueta <session> falsa: Inyecta un token START\_TAG para crear un nuevo elemento <session>, donde el atacante puede definir sus propios atributos.
- Atributo Crítico stageDir: Inyecta sessionStageDir con el valor /data/system.
- Finaliza el documento: Inserta un token END\_DOCUMENT, haciendo que el parser ignore el resto del archivo original y evitando errores de parsing.

### 3.4 La Primitiva de Escritura

Después de que el sistema lee el archivo install\_sessions.xml corrupto, la sesión de instalación modificada se materializa en memoria con un atributo stageDir que apunta a /data/system. Al llamar a los métodos openRead() y openWrite() de esta sesión manipulada, la aplicación maliciosa puede, a través del system\_server, leer y escribir archivos en el directorio /data/system.

### 3.5 La Escalada a SYSTEM

Con la capacidad de escribir en /data/system, el exploit se dirige a su objetivo final: /data/system/packages.xml. El proceso es el siguiente:

1. **Preparación:** Utilizando la primitiva de escritura, el exploit crea un archivo vacío en /data/system/packages-backup.xml.
2. **Lectura y Modificación:** Lee el contenido de packages.xml a través de openRead() y genera una nueva versión del archivo en la que se añade el atributo sharedUserId="android.uid.system" a la entrada correspondiente a su propio paquete.
3. **Escritura:** Escribe esta versión modificada del archivo en /data/system/packages.xml.
4. **Reinicio y Activación:** El exploit fuerza un reinicio del system\_server (y por tanto, del sistema). Al reiniciar, el sistema lee el packages.xml modificado, asigna a la aplicación el UID 1000 (correspondiente al usuario system) y completa el escalado de privilegios.

### 3.6 El Crash Final: Señal SIGSEGV

Para forzar el reinicio, el exploit utiliza la función crashSystemServer(), que envía un comando malicioso a AlarmManager con parámetros corruptos (como la creación de un PooledStringWriter con datos inconsistentes). Esto provoca una violación de segmento en el system\_server, generando la señal SIGSEGV con código SEGV\_ACCERR, que indica que la dirección de memoria existía pero los permisos de acceso no eran los correctos.

## 4. Herramientas para la Recreación

### 4.1 Emulador de Android Studio

- Dispositivo: Pixel 6 (o cualquier modelo compatible)
- Imagen del sistema: Android 13 (API 33) - "Google APIs" (no Google Play)
- Nivel de parche de seguridad: 2024-03-01 (anterior a octubre de 2024)
- Arquitectura: x86\_64

### 4.2 ADB (Android Debug Bridge)

- Ubicación: C:\Users\[usuario]\AppData\Local\Android\Sdk\platform-tools\
- Comandos básicos: adb devices, adb shell, adb root, adb pull, adb install

### 4.3 AndroidQF (Android Quick Forensics)

- Repositorio: <https://github.com/mvt-project/androidqf>
- Instalación: pip install androidqf
- Uso básico: androidqf -o ./extraccion/

### 4.4 MVT (Mobile Verification Toolkit)

- Repositorio: <https://github.com/mvt-project/mvt>
- Instalación en Kali Linux: pip3 install mvt
- Entorno virtual: python3 -m venv ~/mvt-env && source ~/mvt-env/bin/activate

### 4.5 Repositorio del Exploit

- AbxOverflow: <https://github.com/michalbednarski/AbxOverflow>

## 5. Proceso de Recreación del Ataque

Para la realización de la recreación se utilizó un sistema Windows 10 con WSL en conjunto con una máquina virtual Kali Linux.

### 5.1 Preparación del Entorno

Paso 1: Clonar el repositorio del exploit con:

```
mkdir AndroidExploits
cd D:\AndroidExploits
git clone https://github.com/michalbednarski/AbxOverflow.git
```

## Paso 2: Abrir el proyecto en Android Studio

1. Abrir Android Studio y seleccionar "Open".
2. Navegar a D:\AndroidExploits\AbxOverflow y abrir el proyecto.
3. Esperar a que Gradle sincronice las dependencias.

## Paso 3: Configurar las versiones correctas de herramientas

Debido a que el proyecto fue creado en 2024, fue necesario ajustar las versiones de ciertas herramientas para evitar conflictos de compatibilidad:

- Java 11: El proyecto requiere Java 11. En Android Studio, ir a File → Project Structure → SDK Location y configurar Gradle JDK a Java 11.
- Gradle 7.5: Verificar en gradle/wrapper/gradle-wrapper.properties que la distribución sea gradle-7.5-bin.zip.
- SDK 33 (Android 13): Configurar en app/build.gradle el compileSdk 33 y targetSdk 33.

## Paso 4: Compilar el APK

En Android Studio, seleccionar Build → Build Bundle(s) / APK(s) → Build APK. El APK se generará en app/build/outputs/apk/debug/.

## 5.2 Configuración del Emulador Vulnerable

### Paso 1: Crear el emulador

1. En Android Studio, abrir el AVD Manager (View → Tool Windows → Device Manager).
2. Hacer clic en "Create device" y seleccionar un Pixel 6.
3. En la selección de imagen del sistema, elegir Android 13 (API 33) con "Google APIs" (no Google Play).
4. Verificar que el nivel de parche de seguridad sea 2024-03-01 (anterior a octubre de 2024).

### Paso 2: Iniciar el emulador y verificar conectividad ADB con:

```
adb devices
adb shell getprop ro.build.version.security_patch
```

La salida debe mostrar 2024-03-01.

### Paso 3: Conceder permisos de root al ADB con:

```
adb root
```



## 5.3 Ejecución del Exploit

Paso 1: Instalar la aplicación maliciosa con

```
adb install app-debug.apk
```

Paso 2: Ejecutar el exploit

1. Abrir la aplicación "ABX Overflow" en el emulador.
2. Presionar el botón "DO EVERYTHING", que automatiza las etapas Stage 1 (inyección), Stage 2 (modificación de packages.xml) y el crash del system\_server.

Paso 3: Observar el crash y reinicio del sistema

El emulador mostrará un mensaje de que "System UI se detuvo" o similar. El sistema se reiniciará automáticamente. Este crash es intencional y forma parte del mecanismo del exploit.

## 5.4 Verificación de la Escalada de Privilegios

Paso 1: Identificar el proceso de la aplicación maliciosa

```
adb shell pidof com.michalbednarski.abxoverflow
```

Paso 2: Verificar el UID del proceso

```
adb shell cat /proc/[PID]/status | findstr Uid
```

De esto esperamos ver algo como "Uid: 1000 1000 1000 1000"

Paso 3: Verificar el payload instalado

```
adb shell pm list packages | findstr abxoverflow
```

Deberían aparecer los paquetes:

- com.michalbednarski.abxoverflow (aplicación principal)
- com.example.com.michalbednarski.abxoverflow.droppedapk (payload)

## 6. Proceso Forense

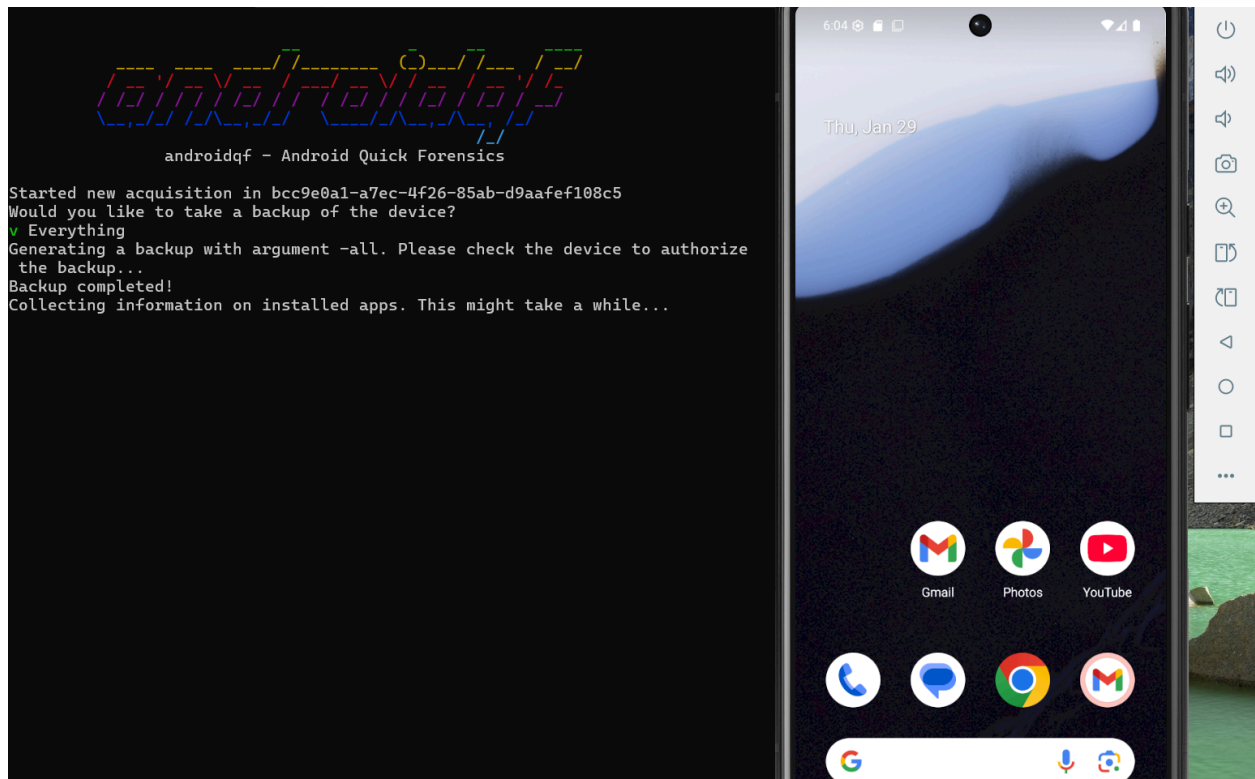
En esta sección describiremos el procedimiento seguido para extraer y analizar las evidencias forenses antes y después de la ejecución del ataque, utilizando las herramientas AndroidQF y MVT.

### 6.1 Extracción con AndroidQF

AndroidQF (Android Quick Forensics) es una herramienta de adquisición forense que permite recolectar de manera ordenada una gran cantidad de archivos del sistema, logs,

configuraciones y listas de aplicaciones de un dispositivo Android, sin requerir conocimientos avanzados de línea de comandos. Para instalarlo y correrlo no hace falta más que seguir la [guía paso a paso aportada por SocialTIC](#)

Así que para iniciar el proceso forense lo que hacemos es hacer una extracción limpia del dispositivo, en nuestro caso el de nuestro emulador previo al ataque para tener una línea de referencia de como se ve un estado sin afectaciones.



El proceso de extracción nos dará una serie de carpetas y archivos que componen a nuestro dispositivo, entre los más relevantes para fines forenses que podemos encontrar están:

- Carpeta de Apks
  - Para ver las aplicaciones instaladas.
- Carpeta de Logs
  - Para ver el registro de eventos dentro del dispositivo.
- Command
  - Registro de comandos utilizados.
- Dumpsys
  - Salida completa del comando dumpsys, que incluye información de diagnóstico de todos los servicios del sistema: actividad, paquetes, ventanas, memoria, batería, etc.
- Logcat

- Registro completo de los logs del sistema Android, incluyendo mensajes del kernel, del framework y de las aplicaciones.
- Bugreport.zip
  - Un reporte de errores completo del dispositivo, que incluye logs del sistema, información de hardware, estado de los servicios y archivos de configuración.
- Hashes.csv
  - Lista en formato CSV con los hashes SHA-256 de cada archivo, garantiza la integridad de la evidencia.

Después de realizar esta extracción limpia y comprender qué archivos pueden ser de interés a futuro pasamos a realizar el ataque sobre nuestro dispositivo tal y como fue explicado en el apartado *Proceso de recreación del ataque*. Y con esto podemos proseguir con hacer una nueva extracción sobre el dispositivo afectado por el ataque.

## 6.2 Analisis con MVT

MVT (Mobile Verification Toolkit) es una herramienta de análisis forense desarrollada por Amnesty International. Su función es procesar los archivos generados por AndroidQF y analizarlos en busca de Indicadores de Compromiso (IOCs), así como estructurar la información para facilitar la detección manual de anomalías. Los pasos que seguí para hacer usar esta herramienta fueron los siguientes:

### Paso 1: Transferir las extracciones a Kali Linux

Debido a que hice mi extracción desde mi máquina Windows las extracciones se tuvieron que transferir a una máquina virtual Kali Linux a través de una carpeta compartida (/media/sf\_KaliCompartido/).

### Paso 2: Activar el entorno virtual de MVT

Usando los comandos en terminal que se detallan a continuación:

```
source ~/mvt-env/bin/activate
```

### Paso 3: Analizar la extracción pre-ataque

```
mvt-android check-androidqf --output ./ analisis_antes ./Limpio/
```

### Paso 4: Analizar la extracción post-ataque

```
mvt-android check-androidqf --output ./ analisis_vulnerado ./Vulnerado/
```

Tras realizar estos comandos notaremos que MVT crea una lista de archivos, algunos nuevos algunos conocidos, pero entre los más notables están los que llevan por nombre “\_detected” que son excerptos del archivo que se nombra como ellos donde se vieron anomalías notables, y mientras que eso nos ayuda a delimitar mejor que buscar también hemos de dar con hallazgos de nuestra propia mano, tal como se verá en el siguiente apartado.

```
(mvt-env)-(kali@kali)-[/media/sf_KaliCompartido]
$ ls ./analysis_vulnerado
aqf_files.json          dbinfo.json            dumpsys_get_prop.json  root_binaries.json
aqf_get_prop.json      dumpsys_activities.json dumpsys_packages.json  timeline.csv
aqf_packages_detected.json dumpsys_adb_state.json dumpsys_platform_compat.json timeline_detected.csv
aqf_packages.json      dumpsys_appops_detected.json dumpsys_receivers.json  tombstones.json
aqf_settings.json      dumpsys_appops.json    info.json
bugreport_timestamps.json dumpsys_battery_daily.json mounts.json
command.log            dumpsys_battery_history.json root_binaries_detected.json
```

## 7. Hallazgos del Análisis Forense

Para empezar, los archivos más notorios que hemos de revisar son:

- **command.log:** Es el registro completo de la ejecución de MVT. Contiene todas las advertencias (WARNING), información (INFO) y errores (ERROR) que la herramienta generó durante el análisis.
- **tombstones.json:** Contiene los volcados de memoria (tombstones) del sistema, que se generan cuando un proceso crítico crashea. En este archivo podemos identificar la fecha del crash, el proceso afectado, su UID (identificador de usuario), y la señal de error.
- **aqf\_packages.json** – Es la lista completa de todos los paquetes (aplicaciones) instalados en el dispositivo. Incluye información como el nombre del paquete, el UID asignado, si es de sistema o de terceros, la ruta de instalación, y los certificados de firma. Permite identificar aplicaciones no oficiales o maliciosas instaladas en el dispositivo.
- **aqf\_packages\_detected.json** – Es un extracto del archivo anterior que contiene únicamente los paquetes que MVT marcó como "no pertenecientes al sistema" o "instalados por métodos no oficiales". Facilita la identificación rápida de aplicaciones sospechosas sin tener que revisar todo el listado completo.
- **bugreport\_timestamps.json** – Contiene las marcas de tiempo de archivos del sistema, incluyendo perfiles de aplicaciones (.prof), tombstones, y archivos ANR (Application Not Responding). Permite reconstruir la cronología del ataque al identificar cuándo se instalaron o ejecutaron las aplicaciones maliciosas y cuándo ocurrieron los crashes del sistema.
- **dumpsys\_appops\_detected.json** – Contiene información sobre el uso de permisos por parte de las aplicaciones, extraída del comando `dumpsys appops`. En este archivo podemos identificar qué aplicaciones accedieron a permisos sensibles y en qué momento. Es útil para detectar actividad anómala de procesos como la shell del sistema.

Si procedemos a ver el contenido de cada uno de ellos daremos con los siguientes resultados.

- Archivo: command.log (Registro de Ejecución)

#### Hallazgo 1.1 – Paquetes no oficiales detectados

El módulo AQFPackages encontró paquetes instalados por métodos no oficiales (ADB o sideloading), lo cual constituye una anomalía significativa:

```
id.modules.androidqf.aqf_packages - WARNING - Found a non-system package installed via adb or another method: "com.michalbednarski.abxoverflow"  
id.modules.androidqf.aqf_packages - WARNING - Found a non-system package installed via adb or another method: "com.michalbednarski.com.michalbednarski.abxoverflow"  
id.modules.androidqf.aqf_packages - WARNING - Found a non-system package installed via adb or another method: "com.example.com.michalbednarski.abxoverflow.droppedapk"
```

Estos paquetes pueden corresponder a alguna aplicación maliciosa. Su presencia es el indicador más claro de compromiso.

#### Hallazgo 1.2 – Configuración de seguridad debilitada

El módulo AQFSettings reportó una configuración global sospechosa:

```
id.modules.androidqf.aqf_settings - WARNING - Found suspicious "global" setting "verifier_verify_adb_installs = 0"  
id.modules.androidqf.aqf_files - INFO - Running module AQFFiles...
```

Esta configuración permite la instalación de aplicaciones desde orígenes desconocidos sin la verificación de seguridad de Google Play, facilitando la instalación de malware.

#### Hallazgo 1.3 – Binarios de root detectados

El módulo RootBinaries encontró binarios de root en el sistema:

```
id.modules.androidqf.root_binaries - WARNING - Found root binary "su" at path "/system/xbin/su"  
id.modules.androidqf.root_binaries - WARNING - Device shows signs of rooting with 1 root binaries found
```

La presencia de su indica que el dispositivo ha sido rooteado o se ejecuta en un emulador con privilegios elevados.

- Archivo: tombstones.json (Volcados de Memoria)

#### Hallazgo 2.1 – Crash del system\_server (tombstone\_00)

Se identificó un tombstone correspondiente al proceso system\_server (PID 559) con las siguientes características:

```

"command_line": [
  "system_server"
],
"pid": 559,
"tid": 16368,
"process_name": "SoundPool_2",
"binary_path": "system_server",
"selinux_label": null,
"uid": 1000,
"signal_info": {
  "code": 2,
  "code_name": "SEGV_ACCERR",
  "name": "SIGSEGV",
  "number": 11
},

```

El proceso system\_server experimentó una violación de segmento (SIGSEGV) con código SEGV\_ACCERR, indicando que intentó acceder a una dirección de memoria existente pero sin los permisos adecuados.

- Archivo: aqf\_packages\_detected.json (Paquetes Instalados encontrados por MVT)

### Hallazgo 3.1 – Aplicación maliciosa principal

```

{
  "name": "com.michalbednarski.com.michalbednarski.abxoverflow",
  "files": [
    {
      "path": "/data/app/~/yze30lnl-6MH00MdDd04rw==/com.michalbednarski.com.michalbednarski.abxoverflow-77iK6YIHykQT-9rnJwbkSw==/base.apk",
      "local_name": "",
      "md5": "3a2cd3a773ee639d4506e10a22fcd3c",
      "sha1": "e02fc83d91b5e2857b9758e18b3a10caead363cf",
      "sha256": "1bc96c26c5db2304b045f1c164ab53bcc195b459ec9d9a88506a2acb2ac2daa4",
      "sha512": "a2d97e358945b7b2b49f0d439d11599539c4d0a2b4f485e890666cec7db26aecdebac19d9cccaeb860f0b02cf28204dfe9302eeb286c743c2076a9d1e12b1b46",
      "error": "",
      "verified_certificate": true,
      "certificate": {
        "md5": "f2b06edfc2d0a01378e02e3f50cdc297",
        "sha1": "0598a6cb358d9caf3a2dae0813b9357b79ce0888",
        "sha256": "0427423d20d2fbc8045b08ff8d5c1ed31d9161c885c55b55d0cd8bc3e5643c72",
        "ValidFrom": "2026-02-12T18:40:41Z",
        "ValidTo": "2056-02-05T18:40:41Z",
        "Issuer": "C=US, O=Android, CN=Android Debug",
        "Subject": "C=US, O=Android, CN=Android Debug",
        "SignatureAlgorithm": "SHA256-RSA",
        "SerialNumber": 1
      },
      "certificate_error": "",
      "trusted_certificate": false
    }
  ],
  "installer": "null",
  "uid": 10175,
  "disabled": false,
  "system": false,
  "third_party": true
},

```

Se identificó el paquete como una aplicación de terceros (no del sistema) instalada en el dispositivo. El uid: 10174 corresponde a una aplicación de usuario normal

## Hallazgo 3.2 – Payload malicioso con privilegios SYSTEM

```
{
  "name": "com.example.com.michalbednarski.abxoverflow.droppedapk",
  "files": [
    {
      "path": "/data/app/~5K_LZ9zdTdABGpDvZ6Mig==/com.example.com.michalbednarski.abxoverflow.droppedapk-UQfoFEYwBC6kxW7-DCGw7w==/base.apk",
      "local_name": "",
      "md5": "403e506df8ca28aea36201e99bc53642",
      "sha1": "e5583703c45933ecb7faf72e003b66b3ffe36193",
      "sha256": "bd2a56621c2c7f3c3772ad16dc883935c0b8195c01dff23a310156313114d8fe",
      "sha512": "33cb0b1848d8a3c2182aa0eff565d479ccaa82e282f94affa4b2968945a6d08c27717fbbe764986c2fe4cd69ae055b079b8f9f344471bf6365c24103d94864c1",
      "error": "",
      "verified_certificate": true,
      "certificate": {
        "Md5": "52ec0f1e4999e274d90f33736daa0fc6",
        "Sha1": "1217d430700bdcc14b38ea6b4bed53204870675d",
        "Sha256": "d8a64e1c3a1afdc340c4b86feaabc88e2d81d66972afbd58e743b7c5b8d1cbdb",
        "ValidFrom": "2023-10-20T08:55:56Z",
        "ValidTo": "2048-10-13T08:55:56Z",
        "Issuer": "CN=abxdroppedapk",
        "Subject": "CN=abxdroppedapk",
        "SignatureAlgorithm": "SHA256-RSA",
        "SerialNumber": 1
      },
      "certificate_error": "",
      "trusted_certificate": false
    }
  ],
  "installer": "null",
  "uid": 1000,
  "disabled": false,
  "system": false,
  "third_party": true
}
```

El paquete tiene uid: 1000, que corresponde al usuario system en Android. Esto demuestra que el exploit logró su objetivo: instalar un payload que se ejecuta con los máximos privilegios del sistema.

- Archivo: bugreport\_timestamps.json (Marcas de Tiempo del Sistema)

## Hallazgo 4.1 – Perfiles de aplicación de las apps maliciosas

Se encontraron marcas de tiempo para los perfiles de las aplicaciones maliciosas:

```
{
  "path": "FS/data/misc/profiles/cur/0/com.michalbednarski.com.michalbednarski.abxoverflow/primary.prof",
  "modified_time": "2026-03-20 07:30:40.000000"
},
{
  "path": "FS/data/misc/profiles/cur/0/com.example.com.michalbednarski.abxoverflow.droppedapk/primary.prof",
  "modified_time": "2026-03-20 07:32:32.000000"
},
}
```

Los perfiles de aplicación (.prof) de las dos aplicaciones maliciosas están presentes en el sistema. La secuencia de marcas de tiempo (07:30:40 y 07:32:32) sugiere que las aplicaciones fueron instaladas y ejecutadas en un orden específico, consistente con el flujo del ataque.

#### Hallazgo 4.2 – Tombstones generados durante el ataque

```
{
  "path": "FS/data/tombstones/tombstone_00.pb",
  "modified_time": "2026-03-05 16:26:50.000000"
},
{
  "path": "FS/data/tombstones/tombstone_00",
  "modified_time": "2026-03-05 16:26:50.000000"
},
{
```

La presencia de estos tombstones confirma que el sistema experimentó un crash durante el período del ataque.

#### Hallazgo 4.3 – Archivos ANR (Application Not Responding)

Se encontraron archivos ANR generados durante el ataque, incluyendo:

```
{
  "path": "FS/data/anr/anr_2026-03-20-07-30-43-523",
  "modified_time": "2026-03-20 07:30:44.000000"
},
{
  "path": "FS/data/anr/anr_2026-03-20-07-29-40-921",
  "modified_time": "2026-03-20 07:30:00.000000"
},
{
```

Los archivos ANR indican que el sistema experimentó bloqueos de aplicaciones durante el ataque.

## 8. Discusión y Conclusiones

Teniendo en cuenta los hallazgos encontrados los siguientes pasos en el proceso forense es el de identificar los indicadores de compromiso notables y construir una línea del tiempo para empezar a unir las piezas del mismo.

### 8.1 Indicadores de Compromiso (IoCs)

Los siguientes son los Indicadores de Compromiso identificados durante el análisis forense. Constituyen las "huellas digitales" dejadas por el atacante en el sistema.

- Indicadores de Archivo y Sistema



- Paquetes (APKs) maliciosos:
  - com.michalbednarski.com.michalbednarski.abxoverflow (Aplicación principal del exploit)
  - com.example.com.michalbednarski.abxoverflow.droppedapk (Payload con privilegios SYSTEM)
- Archivos de sistema modificados:
  - /data/system/packages.xml (Modificado para incluir sharedUserId="1000" para el payload malicioso)
  - /data/system/install\_sessions.xml (Inyectado con entradas de sesión que apuntan a /data/system y /data/app/dropped\_apk)
- Indicadores de Proceso
  - system\_server (PID 559 y otros) experimentando una falla de tipo SIGSEGV.
  - Procesos del payload ejecutándose con UID 1000.

## 8.2 Timeline

| Hora (Aprox.) | Evento                                        | Fuente de Evidencia                                                | Descripción del Evento                                                                                                                      |
|---------------|-----------------------------------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 07:30:40      | Instalación y/o ejecución de la app principal | bugreport_timestamps.json (perfil com.michalbednarski.abxoverflow) | El sistema registra la modificación del perfil de la aplicación, lo que indica que el paquete malicioso primario fue instalado y ejecutado. |
| 07:31:02      | Crash del system_server (1)                   | tombstones.json (tombstone_16)                                     | El proceso del núcleo del sistema operativo falla debido a una violación de segmento (señal SIGSEGV, código SEGV_ACCERR).                   |

|          |                                                     |                                                                                      |                                                                                                                                                                                                  |
|----------|-----------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|          |                                                     |                                                                                      | Este es un evento crítico esperado del exploit.                                                                                                                                                  |
| 07:31:48 | Crash del subsistema de radio (RIL)                 | tombstones.json (tombstone_19)                                                       | El proceso libgoldfish-rild (Radio Interface Layer) también se ve afectado y crashea (SIGSEGV).                                                                                                  |
| 07:32:32 | Instalación y ejecución del payload con privilegios | bugreport_timestamps.json (perfil de droppedapk) y dumpsys_package_s.json (UID 1000) | Se ejecuta la aplicación secundaria (droppedapk). El análisis posterior revela que este proceso se ejecuta con el UID 1000 (usuario system), confirmando el éxito de la escalada de privilegios. |

Conociendo estos datos y apoyándonos del marco de referencia de Mitre Att&ck podemos dar con las Tácticas,técnicas y procedimientos (TTPs) utilizados en el ataque.

### 8.3 Tácticas, Técnicas y Procedimientos (TTPs)

El ataque se puede descomponer en las siguientes tácticas y técnicas según el marco MITRE ATT&CK para dispositivos móviles.

| Táctica                 | ID     | Técnica                                 | ID    | Procedimiento<br>(Procedure) observado                                                                                                                                                                                                               |
|-------------------------|--------|-----------------------------------------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Execution               | TA0041 | Compromise<br>Application<br>Executable | T1577 | El adversario desplegó su código malicioso compilado en un APK (AbxOverflow), que fue instalado en el dispositivo objetivo, permitiendo la ejecución del exploit.                                                                                    |
| Discovery               | TA0032 | File and<br>Directory<br>Discovery      | T1420 | Una vez en ejecución, el exploit leyó el contenido del archivo <code>/data/system/packages.xml</code> para comprender su estructura y poder inyectar su propio registro malicioso.                                                                   |
| Privilege<br>Escalation | TA0029 | Process<br>Injection                    | T1631 | El exploit utilizó la vulnerabilidad CVE-2024-34740 para inyectar una sesión de instalación maliciosa en la memoria del proceso <code>system_server</code> , lo que finalmente le permitió ejecutar su payload con privilegios de SYSTEM (UID 1000). |

## 8.4 Conclusiones

El análisis de los archivos generados por MVT permitió identificar de manera clara y estructurada los rastros de un ataque con escalación de privilegios. Las principales evidencias siendo:

- La existencia de los paquetes maliciosos
- La confirmación del payload con privilegios SYSTEM (uid: 1000)
- El crash del system\_server con señal SIGSEGV
- Las marcas de tiempo de los perfiles de aplicación
- Los tombstones y archivos ANR

Todos estos hallazgos, en conjunto, permiten reconstruir la cadena de ataque completa y confirmar la explotación exitosa de la vulnerabilidad CVE-2024-34740. La práctica demostró que:

- El emulador con Android 13 y parche de seguridad 2024-03-01 es vulnerable al exploit.
- El exploit logra escalar privilegios de una aplicación normal a SYSTEM (UID 1000).
- Las herramientas forenses como AndroidQF y MVT, complementadas con análisis manual, son capaces de detectar los rastros del ataque.

Estos resultados confirman la importancia de mantener los dispositivos actualizados con los parches de seguridad más recientes, así como la utilidad de las herramientas forenses para la detección de compromisos en sistemas Android.

## 9. Referencias

[1] M. Bednarski, "AbxOverflow," GitHub. [Online]. Available: <https://github.com/michalbednarski/AbxOverflow>. [Accessed: 2026].

[2] Amnesty International, "Mobile Verification Toolkit (MVT)," GitHub. [Online]. Available: <https://github.com/mvt-project/mvt>. [Accessed: 2026].

[3] Amnesty International, "Android Quick Forensics (AndroidQF)," GitHub. [Online]. Available: <https://github.com/mvt-project/androidqf>. [Accessed: 2026].

[4] SocialTIC, "Guías de extracción forense," [Online]. Available: <https://forensics.socialtic.org/how-tos/index.html>. [Accessed: 2026].

[5] MITRE, "MITRE ATT&CK for Mobile," [Online]. Available: <https://attack.mitre.org/techniques/mobile/>. [Accessed: 2026].